

CORA: Analyzing and bridging thinking-answer gap in Multimodal RLVR via Consistency-Oriented Reasoning Alignment

Jiayue Cao^{1*}, Zhicong Lu^{1*}, Xuehan Sun², Wei Jia^{1†}, Hongling Zheng²
Changyuan Tian¹, Zichuan Lin³, Wenqian Lv¹, Nayu Liu⁴

¹University of Chinese Academy of Sciences

²Wuhan University, ³Tsinghua University, ⁴Tianjin University

Abstract

Reinforcement learning with verifiable rewards (RLVR) has successfully elicited the reasoning capabilities of large language models, motivating its extension to multimodal scenarios. Existing methods primarily focus on improving the visual coverage of reasoning traces and mitigating visual hallucinations, but underestimate the semantic inconsistency between the reasoning process and the final answer. In this paper, we delve into thinking-answer inconsistency in RLVR for large vision-language models (LVLMs), showing thorough analyses of rollouts collected throughout Group Relative Policy Optimization (GRPO) training process and post-RLVR evaluation outputs that this issue persists during training and remains present during inference. Motivated by the analysis, we propose **Consistency-Oriented Reasoning Alignment (CORA)**, which introduces thinking-answer semantic consistency into RLVR through a lightweight plug-and-play consistency reward model, and further incorporates Hybrid Reward Advantage Splitting (HRAS) to stably coordinate task and consistency optimization. Extensive experiments across representative multimodal reasoning benchmarks and mainstream LVLMs show that CORA improves task performance while effectively mitigating thinking-answer inconsistency, leading to more faithful reasoning traces.

1 Introduction

Reinforcement learning with verifiable rewards (RLVR) has recently shown strong effectiveness in enhancing the reasoning capabilities of large language models (Guo et al., 2025; Chen et al., 2026; Jia et al., 2025; Diao et al., 2026; Lu et al., 2025). Building on this success, a growing line of work seeks to extend RLVR to large vision-language models (LVLMs), aiming to enhance their ability to

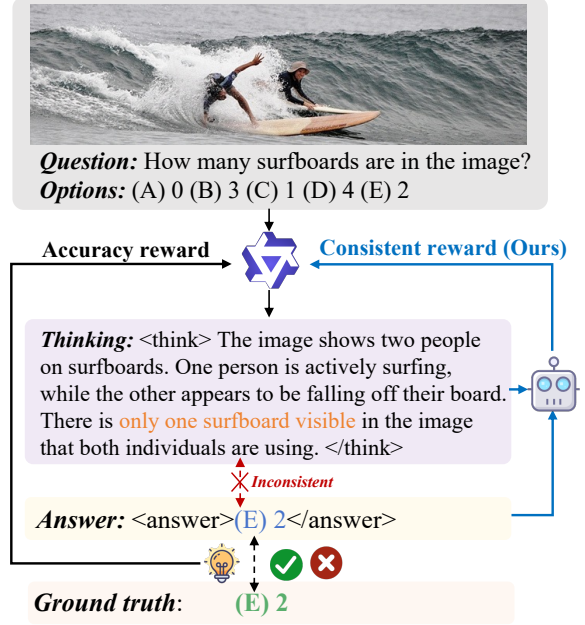


Figure 1: Thinking-answer inconsistency in multimodal RLVR. Existing works typically use final-answer correctness as the accuracy reward, underestimating potential inconsistencies between the reasoning trace and the final answer. Our method introduces the consistency reward to encourage the model to derive final answers from faithful reasoning traces.

perform complex multimodal reasoning (Liu et al., 2025; Huang et al., 2025b; Feng et al., 2026).

Following the standard RLVR paradigm that generates explicit reasoning traces before final answers, previous methods (Man et al., 2025; Lim et al., 2026) primarily focus on improving visual coverages and reducing hallucinations of reasoning traces. However, they rely on a foundational assumption: reasoning traces foster faithful final-answer generation, which can be fragile under answer-level rewards, where reasoning and answer may diverge despite the answer being correct. Concurrent works (Chen et al., 2025; Huang et al., 2025a) either heuristically recognize this reasoning-

*Equal contribution.

†Corresponding author.

answer mismatch within specific task settings or rely on costly expensive auxiliary reward mechanisms, leaving the dynamic evolution of such inconsistency during training insufficiently explored.

To further investigate this issue, we conduct an empirical analysis of on-policy rollouts during group relative policy optimization (GRPO) (Shao et al., 2024; Lu et al., 2026) training and evaluation. Specifically, we analyze several mainstream open-source LLMs (Wang et al., 2024b; Bai et al., 2025) of different scales throughout the training process, thereby tracing how thinking-answer inconsistency evolves dynamically during RLVR. To ensure the comprehensive analysis, we select common scenarios (Ray et al., 2024; Shi et al., 2024; Lu et al., 2024a; Ghosal et al., 2025) in multimodal reasoning, including spatial reasoning, multimodal mathematical reasoning, and puzzle. We observe that thinking-answer inconsistency is not merely caused by a small number of outlier cases, but widely emerges during RLVR with GRPO. Moreover, this issue consistently persists throughout the training dynamics, rather than being naturally mitigated as RLVR training progresses. In addition, standard GRPO is insufficient to reliably mitigate this issue as training progresses. Even after RLVR, models can still produce reasoning traces that fail to support the final answer or even semantically contradict it during evaluation. We attribute this issue arises from the answer-level reward design, which supervises only the final answer. This may lead the model to learn shortcuts for reaching correct answers, rather than deriving them from the generated reasoning traces.

Motivated by above evaluation observations and analyses, we propose **Consistency-Oriented Reasoning Alignment (CORA)**, which explicitly regularizes the semantic consistency between the thinking process and the answer during RLVR. Specifically, we introduce a lightweight and plug-and-play Consistency Reward Model (CRM) that scores thinking-answer consistency in a Natural Language Inference (NLI)-style discriminative manner, and incorporate this signal into GRPO as an additional reward. Moreover, to properly incorporate a continuous consistency reward while avoiding conflicts with the original discrete accuracy reward under group-wise normalization, we propose Hybrid Reward Advantage Splitting (HRAS), a reward-decoupled advantage estimation strategy that preserves the distinct preference signals of task and consistency rewards through sepa-

rate group-wise normalization and weighted advantage composition.

To validate the effectiveness and generalization of our method, we carry out extensive experiments on three scenarios of multimodal reasoning benchmarks with mainstream LLMs. Compared with RLVR using standard GRPO, our method achieves stronger performance and effectively mitigates the semantic inconsistency between thinking and answers, further demonstrating its effectiveness and generalizability. Overall, our contributions are summarized as follows:

- 1) We conduct an in-depth analysis of thinking-answer inconsistency in RLVR for multimodal reasoning, showing that this issue persists throughout GRPO training and is not naturally mitigated in post-RLVR evaluation.

- 2) We propose CORA, a consistency-oriented RLVR method that introduces a lightweight and plug-and-play consistency reward model to enhance thinking-answer semantic consistency, together with HRAS for stable joint optimization of task and consistency rewards.

- 3) Extensive experiments on multiple representative multimodal reasoning benchmarks demonstrate that CORA consistently improves performance across mainstream LLMs while effectively mitigating thinking-answer inconsistency. The code will be released soon to foster future research in RLVR for multimodal reasoning.

2 Thinking-Answer Inconsistency in RLVR

In this section, we first define a binary consistency measure to evaluate the degree of consistency between thinking and answer during RLVR-based training and inference. We then conduct a series of exploratory experiments (Lu et al., 2024b) to investigate whether LLMs trained with answer-correctness rewards exhibit inconsistency between their thinking process and final answer. Finally, we summarize the quantitative findings and provide qualitative analyses of this issue.

2.1 Thinking-Answer Consistency Measure

Given a question q , a reasoning process t within the `<think>` tag, and a final answer a within the `<answer>` tag, we define thinking-answer consistency as a binary judgment. A sample is labeled as *consistent* if the thinking t semantically supports or leads to the answer a ; otherwise, it is labeled

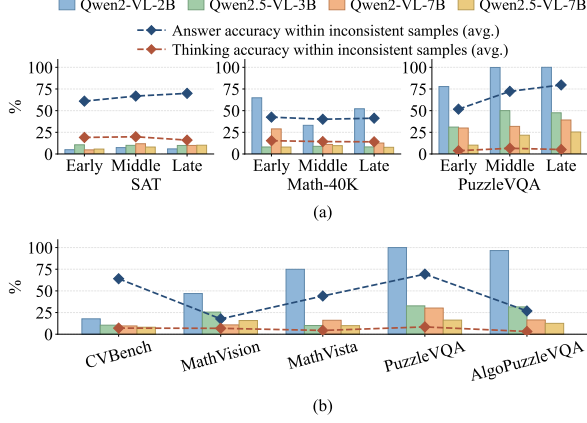


Figure 2: Comparison of inconsistency rate and thinking-vs-answer accuracy among inconsistent samples. (a) Phenomenon during training. (b) Phenomenon on evaluation benchmarks.

as *inconsistent*. Inconsistent cases include: (1) the answer implied by the thinking contradicts the final answer; and (2) the reasoning process fails to provide substantive evidence for the final answer. We use the Inconsistency Rate (IR) to quantify the degree of thinking-answer inconsistency during training and inference.

$$IR = \frac{N_{\text{incons}}}{N_{\text{valid}}}, \quad (1)$$

where N_{incons} denotes the number of samples labeled as inconsistent, and N_{valid} denotes the total number of valid samples whose thinking and answer can be extracted and evaluated. To investigate whether the generated thinking reliably supports the final answer during RLVR-based training and inference, we conduct the following exploratory experiments.

2.2 Empirical Analysis

Data. We focus on three representative multimodal reasoning tasks: visual perception, including classification and spatial grounding; multimodal mathematical reasoning; and visual puzzle reasoning. For visual perception, we train on the SAT dataset (Ray et al., 2024) and evaluate on CVBench dataset (Tong et al., 2024). For multimodal mathematical reasoning, we train on Math-40K (Shi et al., 2024) and evaluate on MathVision (Wang et al., 2024a) and MathVista (Lu et al., 2024a). For visual puzzle reasoning, we follow the setting of (Li et al., 2026) and construct a 6.5K-sample PuzzleVQA training set using the generation code released by (Chia et al., 2024). During evaluation,

we use PuzzleVQA as the in-domain test set and AlgoPuzzleVQA (Ghosal et al., 2025) as the out-of-domain test set.

Experimental Setup. To investigate whether thinking and answers remain consistent in multimodal reasoning, we conduct a preliminary study under a standard RLVR setting. We train four Qwen-series LVLMs on the three representative multimodal reasoning tasks described above, using format compliance and answer correctness as reward signals. The experimental details are kept consistent with the main experiments and are provided in Section 4.1.

After training, we construct two corpora for consistency analysis. For the training-stage corpus, model states from the early, middle, and late stages of each training run are selected and used to generate responses on the corresponding training prompts. We then extract the thinking and answer contents from each response, resulting in a training-stage thinking-answer corpus with approximately 719K valid samples. For the evaluation-stage corpus, the final trained model from each run is used to generate responses on evaluation benchmarks, and the corresponding thinking-answer pairs are extracted in the same manner.

To annotate sample-level consistency in the corpus, we design a dedicated judging prompt, which is provided in the Appendix D. The prompt instructs the judge model to perform three steps for each response: (1) extract the final conclusion from the reasoning traces within the <think> tag; (2) extract the final answer from the <answer> tag; and (3) determine whether the thinking process and the final answer point to the same semantic answer. The annotation rules are as follows: if the conclusion from the thinking process is semantically equivalent to the final answer, the consistency label is *Yes*; if they point to different values, the label is *No*; and if either the reasoning process or the final answer does not contain an extractable answer, the label is *None*. The first case is treated as consistent, while the latter two cases are treated as inconsistent. Across all datasets, a unified annotation template is adopted, with dataset-specific few-shot examples provided to accommodate different task formats.

During annotation, GPT-5 (OpenAI, 2025) is used as the judge model. Based on these annotations, we quantify whether the thinking process and final answer remain consistent during RLVR training and final inference, as well as the extent of inconsistency when it occurs.

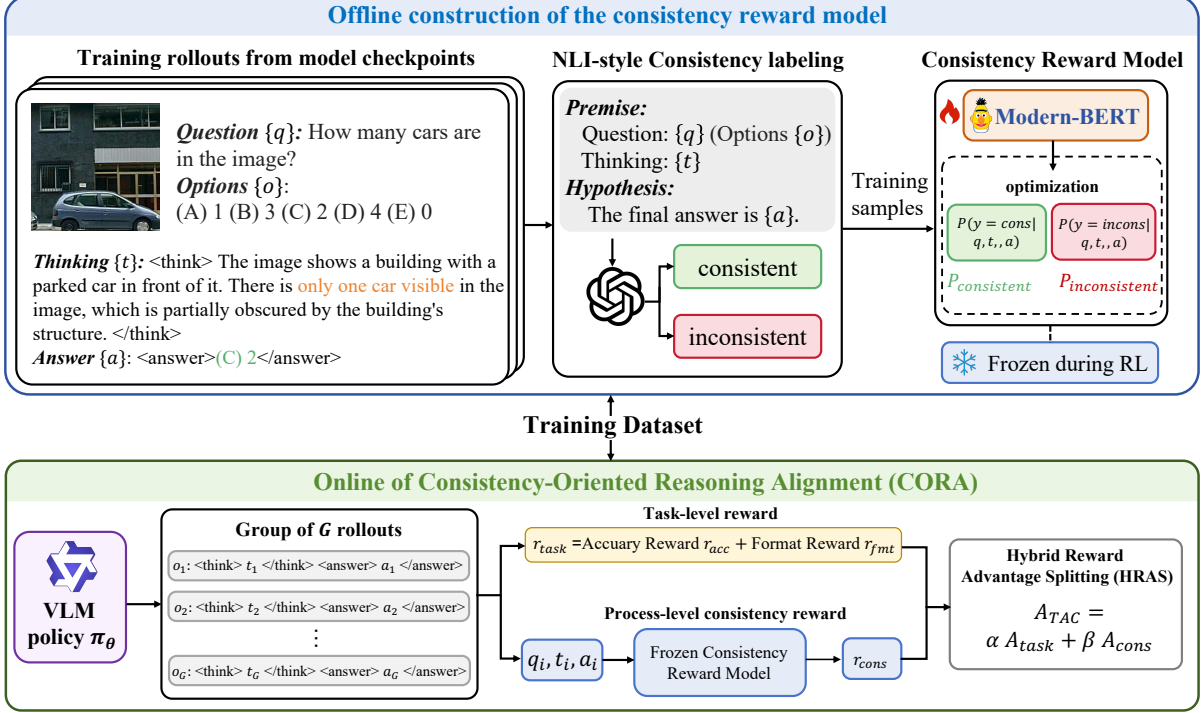


Figure 3: Overview of CORA. We construct an NLI-style consistency discrimination dataset from the training-stage corpus to train a ModernBERT-based CRM. During GRPO, the CRM provides a consistency reward for each generated <think>–<answer> pair. CORA separately normalizes task and consistency rewards into split advantages and combines them for policy optimization.

2.3 Quantitative Results and Analysis

Figure 2 compares thinking-answer inconsistency during training and evaluation. The results show that inconsistency exists in both stages of LVLM behavior and is not an occasional failure case, but a widespread phenomenon across models and visual reasoning tasks. The inconsistency rate is task-dependent: it is lower on visual perception tasks, where answers can often be directly inferred from visual evidence, but higher on multimodal mathematical reasoning and visual puzzle reasoning tasks, which require deeper computation and multi-step inference. Moreover, inconsistency persists throughout training and often becomes more pronounced as training proceeds. The same issue further appears on evaluation benchmarks. Finally, among inconsistent samples, the final answer is more often correct than the reasoning-implied answer. This suggests that current RLVR training primarily optimizes answer correctness while leaving the thinking process under-supervised.

3 Method

In this section, we present the proposed CORA method. As illustrated in Figure 3, CORA employs

the CRM to estimate thinking-answer consistency and integrates it into RLVR as an explicit reward signal. To mitigate interference among heterogeneous rewards, we introduce a reward-advantage splitting strategy that separates the optimization signals from task rewards and consistency rewards.

3.1 Consistency Reward Model

Based on the preliminary analysis, we argue that RLVR with answer-level rewards suffers from a lack of direct supervision over the thinking process. Although such rewards can encourage models to produce correct final answers, they do not explicitly verify whether the reasoning process semantically supports the final output answer. To generate more reliable and consistent reasoning, we introduce an explicit consistency reward into the RLVR process.

Specifically, we formulate the problem of judging thinking-answer consistency as a NLI task. We then construct a labeled dataset and train a lightweight consistency reward model to score, during rollout, whether the generated thinking process is semantically consistent with the final answer. This score is incorporated into training as an additional consistency reward.

NLI-style formulation. Given a question q ,

thinking process t , and answer a , we formulate thinking-answer consistency as a NLI problem. Specifically, we combine q and t as the premise, and treat a as the hypothesis.

$$\begin{aligned} P(q, t) &= [\text{Question: } q; \text{ Thinking: } t], \\ H(a) &= [\text{The final answer is: } a]. \end{aligned} \quad (2)$$

When $P(q, t)$ semantically entails $H(a)$, the thinking and the answer are considered consistent; otherwise, they are considered inconsistent.

Dataset construction. We construct the dataset for training the CRM using the training-stage corpus described in Section 2.2. For each sample in the corpus, we extract the question, thinking process, and final answer, and then convert them into NLI-style premise-hypothesis pairs following the format in Eq. 2. We further perform stratified sampling according to data source, thinking-answer consistency label, and training stage. Finally, we obtain a consistency discrimination dataset containing approximately 90K samples. The dataset is split into training and test sets with an 8:2 ratio. The training set is used to train the CRM, while the test set is used to evaluate its ability to distinguish thinking-answer consistency.

NLI-based Consistency Reward Model. During RLVR training, the CRM needs to be invoked for every rollout to produce a consistency reward, making both inference efficiency and scoring quality important. We adopt ModernBERT (Warner et al., 2025) as a lightweight discriminative encoder for consistency reward prediction. ModernBERT supports long-context inputs of up to 8192 tokens and is well suited for classification tasks, making it appropriate for our NLI-based consistency scoring setting. Compared with using a LLM as an online judge during training, this design provides a more efficient and cost-effective reward function.

We fine-tune ModernBERT on the constructed 90K NLI-style consistency discrimination dataset using a cross-entropy loss. During training, the premise and hypothesis are jointly encoded. The hidden representation of the [CLS] token is then fed into a two-layer classification head to predict a binary label, i.e., consistent or inconsistent. Details of the CRM training hyperparameters and evaluation results are provided in Appendix B.

After training, the CRM is frozen and used as a scoring function during the RLVR stage. The weighted consistency reward is then defined as

$$r_{\text{cons}}(q, t, a) = \lambda_{\text{cons}} \cdot p(y = \text{cons} \mid q, t, a), \quad (3)$$

where λ_{cons} controls the contribution of the consistency reward and y denotes the thinking-answer consistency label.

3.2 Hybrid Reward Advantage Splitting

Standard GRPO samples a group of responses for each prompt and sums all reward components of each response into a single scalar reward. The advantage is then computed by normalizing these summed rewards within the response group. However, different rewards may have different distributional properties, and directly summing them before advantage computation can introduce scale coupling among heterogeneous reward signals. Specifically, format rewards and answer-correctness rewards are typically sparse and discrete, whereas the consistency rewards produced by the CRM are continuous probability scores. When rewards with different distributions are directly added and normalized together, the reward component with larger within-group variance may dominate the final advantage estimate, thereby weakening the advantage signals from other rewards. For consistency-aware RLVR, this leads to a potential issue: if the consistency reward dominates, the model may overemphasize thinking-answer alignment while under-optimizing answer correctness. Conversely, if the answer-correctness reward dominates, the consistency signal may become too weak to effectively constrain the thinking process.

To address this issue, we propose HRAS. HRAS groups the format reward and answer-correctness reward as the task reward, and computes the advantages of the task reward and the consistency reward separately within each response group. The two advantages are then combined with weighted coefficients to form the final advantage, preventing interference between heterogeneous reward signals.

$$A_{\text{TAC}}^{(i)} = \alpha \frac{r_{\text{task}}^{(i)} - \mu_{\text{task}}}{\sigma_{\text{task}} + \epsilon} + \beta \frac{r_{\text{cons}}^{(i)} - \mu_{\text{cons}}}{\sigma_{\text{cons}} + \epsilon}. \quad (4)$$

Here, $\mu_{\text{task}}, \sigma_{\text{task}}$ and $\mu_{\text{cons}}, \sigma_{\text{cons}}$ are computed separately over the G responses sampled for the same prompt, and α, β control the relative strength of task learning and consistency learning.

4 Experiments

4.1 Experimental Setup

Dataset. To evaluate the effectiveness of our method, we conduct RLVR experiments using

Model	Method	CVBench		MathVision		MathVista		PuzzleVQA		AlgoPuzzleVQA	
		Acc↑	IR↓	Acc↑	IR↓	Acc↑	IR↓	Acc↑	IR↓	Acc↑	IR↓
Qwen2-VL-2B	Baseline	68.12	17.77	15.36	46.96	46.10	74.90	51.10	100	29.39	96.55
	CORA	68.39	12.44	11.71	54.78	47.00	55.46	63.35	99.90	28.28	99.09
Qwen2-VL-7B	Baseline	77.52	9.56	17.53	10.76	58.70	16.09	77.00	30.21	26.72	16.46
	CORA	78.09	8.64	18.78	9.71	58.50	15.26	81.95	5.03	26.67	5.31
Qwen2.5-VL-3B	Baseline	76.04	10.43	19.01	25.52	59.40	10.05	65.45	32.71	28.78	31.54
	CORA	76.50	9.11	19.34	24.95	61.30	15.08	64.35	24.06	30.17	30.38
Qwen2.5-VL-7B	Baseline	77.75	8.08	12.99	15.80	67.90	9.89	76.10	16.29	31.28	12.54
	CORA	78.24	7.36	17.00	8.28	69.30	6.48	76.60	3.37	31.83	2.50

Table 1: Main results on five multimodal reasoning benchmarks across four Qwen-VL backbones. Best results are highlighted in **bold**.

GRPO on three scenarios of multimodal reasoning benchmarks, including CVBench, MathVision, MathVista, PuzzleVQA, and AlgoPuzzleVQA, following the setup in Section 2.2. More details of datasets are provided in Appendix A.

Implementations. We use Qwen2-VL-2B/7B-Instruction and Qwen2.5-VL-3B/7B-Instruction as the backbone LVLMs. We perform RLVR on the training datasets from the three multimodal scenarios, respectively. The baseline follows the standard RLVR setting and uses only answer-correctness and format rewards, without the proposed consistency reward. CORA keeps the same task rewards and additionally incorporates the CRM-based consistency reward with HRAS. Across all experimental settings, we set the batch size to 1, use gradient accumulation of 2 steps, and set the format reward weight to 0.1. Training is conducted for two epochs on SAT and PuzzleVQA, and for one epoch on Math-40K. Other detailed training configurations are provided in Appendix A.

4.2 Main Result

Table 1 reports the accuracy and inconsistency rate on five evaluation benchmarks across three multimodal reasoning tasks. Overall, in most settings, CORA substantially reduces the inconsistency between thinking and answer while also improving final-answer accuracy. We further observe that the performance gains brought by CORA are more pronounced on 7B-scale backbones than on smaller 2B/3B models. A possible explanation is that consistency supervision assumes that the model can first generate a meaningful reasoning process. Larger models typically possess stronger reasoning capabilities, making their generated thinking

content more amenable to consistency evaluation and optimization. By contrast, smaller models may generate incomplete or vacuous reasoning traces, thereby limiting the effectiveness of the consistency reward mechanism.

Moreover, the effectiveness of CORA is also task-dependent. The performance gains are more pronounced on MathVision and AlgoPuzzleVQA. This is because both benchmarks require deeper reasoning capabilities, where the final answer depends heavily on the intermediate reasoning process. CORA provides an explicit process-level signal that encourages the generated thinking process to more effectively support the final answer. The reward curves in Appendix E support our claim.

Furthermore, we observe that reductions in inconsistency rate are associated with improvements in final-answer accuracy. This suggests that thinking-answer consistency is closely related to the model’s task-solving ability. When the generated thinking semantically supports the final answer, the final prediction is more likely to arise from a coherent and effective reasoning process. This observation further supports our hypothesis that RLVR training based solely on final-answer correctness may not sufficiently supervise the thinking process, whereas explicitly introducing a consistency reward can encourage the model to generate more reliable reasoning and thereby improve final-answer correctness.

4.3 Ablation Study

To verify the effect of the CRM and HRAS, we perform ablation studies using Qwen2.5-VL-7B as the backbone. When ablating HRAS, we keep the task reward and consistency reward formula-

Method	Metric	CVBench	MathVision	MathVista	PuzzleVQA	AlgoPuzzleVQA
CORA	Acc $\uparrow$	78.24	17.00	69.3	76.6	31.83
	IR $\downarrow$	7.36	8.28	6.48	3.37	2.50
	ConsAcc $\uparrow$	80.03	26.38	73.24	76.56	32.68
w/o CRM	Acc $\uparrow$	77.75	12.99	67.90	76.10	31.28
	IR $\downarrow$	8.08	15.80	9.89	16.29	12.54
	ConsAcc $\uparrow$	80.45	27.22	74.77	74.91	32.17
w/o HRAS	Acc $\uparrow$	76.12	14.08	68.1	75.3	30.78
	IR $\downarrow$	1.22	10.60	0.50	0.55	3.27
	ConsAcc $\uparrow$	80.49	23.90	70.49	75.57	30.90

Table 2: Ablation results of CRM and HRAS on Qwen2.5-VL-7B. ConsAcc denotes the answer accuracy among samples whose thinking process is consistent with the final answer. Best results are highlighted in **bold**.

tions unchanged, and replace Eq. 4 with a conventional single-advantage computation based on the summed reward.

As shown by the answer accuracy results in Table 2, the proposed CORA method achieves the best performance across all five evaluation benchmarks. The w/o CRM and w/o HRAS variants exhibit mixed gains and drops across different benchmarks, but both underperform CORA overall. This suggests that CORA effectively balances reasoning consistency with answer correctness.

Directly adding the consistency reward to the total reward can achieve stronger inconsistency reduction on some benchmarks. However, this variant consistently underperforms CORA in final-answer accuracy. We attribute this trade-off to the training dynamics of merged reward advantage computation. In the later stage of training, responses within the same GRPO group often show limited variation in answer correctness, while their consistency scores may still vary substantially. As a result, the advantage signal produced by merged reward computation can be dominated by the consistency signal. This tendency may drive policy updates away from optimizing answer correctness. The fact that CORA achieves higher ConsAcc than the w/o HRAS variant on most benchmarks provides further supporting evidence.

4.4 Analyses

Reliability Analysis. To analyze whether CORA improves performance by promoting more reliable reasoning, we categorize model responses into four groups according to answer correctness and thinking-answer consistency, as shown in Table 3. We compare the group distributions between the baseline and CORA on MathVision and PuzzleVQA. The results in Table 3 show that CORA

Dataset	Method	CC	CW	IC	IW
MathVision	Baseline	9.08	30.20	1.48	59.24
	CORA	15.46	53.75	1.02	29.77
	Δ pp	+6.38	+23.55	-0.46	-29.47
PuzzleVQA	Baseline	62.55	20.95	12.90	3.60
	CORA	73.55	22.65	2.85	0.95
	Δ pp	+11.00	+1.70	-10.05	-2.65

Table 3: Comparison of sample categorization between the baseline and CORA on MathVision and PuzzleVQA, using Qwen2.5-VL-7B as the backbone. CC denotes Consistent-Correct, CW denotes Consistent-Wrong, IC denotes Inconsistent-Correct, and IW denotes Inconsistent-Wrong.

increases the proportion of responses that are both correct and consistent, while reducing the proportion of correct but inconsistent responses. This suggests that CORA promotes semantically supported correctness, where correct final answers are accompanied by reasoning traces that provide valid justification.

Case study. Figure 4 illustrates how CORA improves the reliability of generated reasoning. The baseline may output a correct answer with an inconsistent reasoning trace, as in the pillow-counting example, or fail both reasoning and prediction, as in the clock example. In contrast, CORA produces reasoning processes that explicitly support the final answers. These cases qualitatively demonstrate that CORA encourages the model to generate answers with semantically aligned and better-justified reasoning traces.

5 Related Works

Multimodal Chain-of-Thought Reasoning. Multimodal chain-of-thought reasoning enables LVLMS to generate intermediate reasoning steps before producing final answers. Early studies

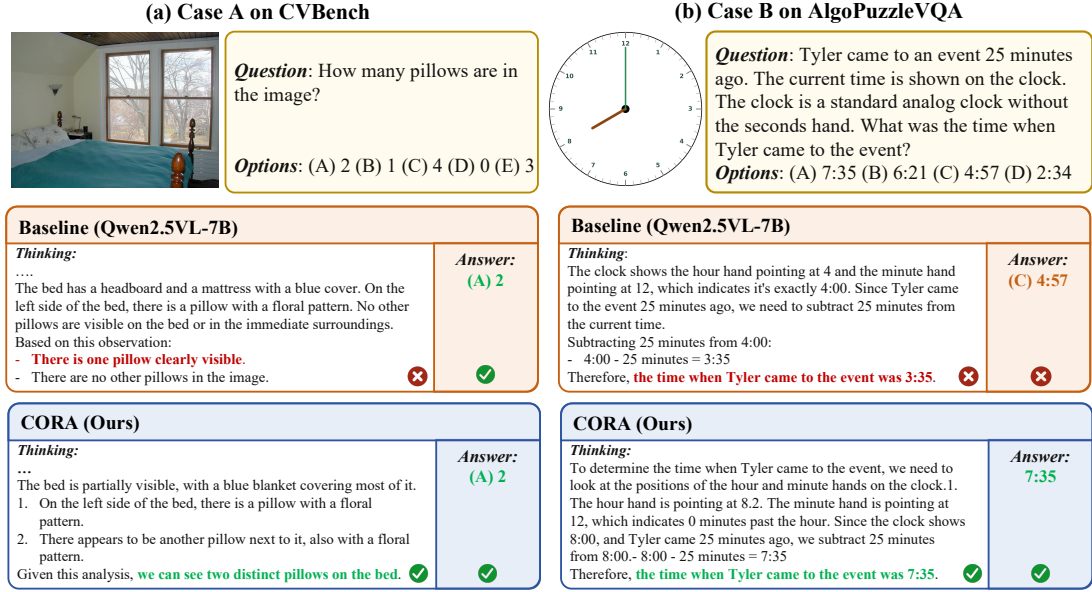


Figure 4: Case studies on different benchmarks.

such as Multimodal-CoT (Zhang et al., 2023) show that generating intermediate rationales can improve performance on multimodal question answering tasks. Recent studies move beyond purely text-based reasoning traces and emphasize the critical role of visual evidence in the reasoning process. Argus (Man et al., 2025) introduces a grounded chain-of-thought reasoning paradigm that assists reasoning through visual attention revisiting. VG-CoT (Lim et al., 2026) explicitly links each reasoning step to visual evidence, further highlighting the importance of trustworthy visual reasoning. These methods often implicitly assume that the reasoning process can be reliably aligned with the final answer, and mainly focus on improving the reasoning process itself. However, in generated responses, the reasoning trajectory is not necessarily consistent with the final answer.

RLVR for Multimodal Reasoning. Reinforcement Learning with Verifiable Rewards (RLVR) has recently emerged as an effective paradigm for improving the reasoning ability of large language models. Inspired by this success, some studies (Tian et al., 2026a,b) have extended RLVR to multimodal reasoning. Visual-RFT (Liu et al., 2025) adapts reinforcement fine-tuning to VLMs by designing verifiable rewards for perception and grounding tasks. Subsequent works further explore RLVR through CoT supervised fine-tuning (Tan et al., 2026), multimodal cold-start data (Huang et al., 2025b), and iterative SFT-RL self-improvement (Deng et al., 2025). Most exist-

ing RLVR methods still primarily optimize rewards defined at the task-output level. Recent studies have begun to recognize this limitation and introduce additional process-oriented signals. However, concurrent works either heavily rely on ground-truth information or are restricted to specific multiple-choice settings (Kan et al., 2025; Huang et al., 2025a), making them difficult to generalize to diverse multimodal reasoning scenarios.

6 Conclusion

In this paper, we delve into thinking-answer inconsistency in RLVR for LVLMs, where the generated reasoning trace may fail to support, or even semantically contradict to the final answer. Through thorough analyses of on-policy rollouts during GRPO training and post-RLVR evaluation outputs, we show that this issue persistently emerges during training and is not naturally mitigated by standard GRPO. To address this problem, we propose CORA, a consistency-oriented RLVR method that introduces a lightweight and plug-and-play consistency reward model to promote thinking-answer consistency. We further design HRAS to decouple task and consistency rewards during advantage estimation, enabling stable joint optimization of answer correctness and reasoning reliability. Extensive experimental results across representative multimodal reasoning benchmarks demonstrate that CORA improves task performance while effectively reducing thinking-answer inconsistency.

Limitations

Despite the impressive results of our method, we have to admit our work has the following limitations: (1) due to computational constraints, we only evaluate the effectiveness of the proposed method on models with 7B parameters or fewer. Its applicability to larger-scale models remains to be further validated. (2) Our current study focuses on image-text multimodal reasoning tasks, while higher-dimensional visual inputs such as videos are not considered. Evaluating the proposed method on broader and more diverse tasks will be an important direction for our future work.

References

- Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibao Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yanzhi Zhu, Ming-Hsuan Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, and 8 others. 2025. Qwen2.5-vl technical report. *CoRR*, abs/2502.13923.
- Qiguang Chen, Libo Qin, Jinhao Liu, Dengyun Peng, Jiannan Guan, Peng Wang, Mengkang Hu, Yuhang Zhou, Te Gao, and Wanxiang Che. 2026. Towards reasoning era: A survey of long chain-of-thought for reasoning large language models. *Science China Information Sciences*, 69(6):161101.
- Yi Chen, Yuying Ge, Rui Wang, Yixiao Ge, Junhao Cheng, Ying Shan, and Xihui Liu. 2025. GRPO-CARE: consistency-aware reinforcement learning for multimodal reasoning. *CoRR*, abs/2506.16141.
- Yew Ken Chia, Vernon Toh, Deepanway Ghosal, Lidong Bing, and Soujanya Poria. 2024. Puzzlevqa: Diagnosing multimodal reasoning challenges of language models with abstract visual patterns. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 16259–16273.
- Yihe Deng, Hritik Bansal, Fan Yin, Nanyun Peng, Wei Wang, and Kai-Wei Chang. 2025. Openvlthinker: An early exploration to complex vision-language reasoning via iterative self-improvement. *arXiv e-prints*, pages arXiv–2503.
- Juncheng Diao, Zhicong Lu, Peiguang Li, Yongwei Zhou, Changyuan Tian, Qingbin Li, Rongxiang Weng, Jingang Wang, and Xunliang Cai. 2026. Hipif: Hierarchical planning and information folding for long-horizon llm agent learning. *Preprint*, arXiv:2606.10507.
- Kaituo Feng, Kaixiong Gong, Bohao Li, Zonghao Guo, Yibing Wang, Tianshuo Peng, Junfei Wu, Xiaoying Zhang, Benyou Wang, and Xiangyu Yue. 2026. Video-r1: Reinforcing video reasoning in mllms. *Advances in Neural Information Processing Systems*, 38:99114–99137.
- Deepanway Ghosal, Vernon Toh, Yew Ken Chia, and Soujanya Poria. 2025. Algotpuzzlevqa: Diagnosing multimodal reasoning challenges of language models with algorithmic multimodal puzzles. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 9615–9632.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, and 1 others. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.
- Minbin Huang, Runhui Huang, Chuanyang Zheng, Jingyao Li, Guoxuan Chen, Han Shi, and Hong Cheng. 2025a. Answer-consistent chain-of-thought reinforcement learning for multi-modal large language models. *arXiv preprint arXiv:2510.10104*.
- Wenxuan Huang, Bohan Jia, Zijie Zhai, Shaosheng Cao, Zheyu Ye, Fei Zhao, Zhe Xu, Xu Tang, Yao Hu, and Shaohui Lin. 2025b. Vision-r1: Incentivizing reasoning capability in multimodal large language models. *arXiv preprint arXiv:2503.06749*.
- Wei Jia, Li Jin, Kaiwen Wei, Yuying Shang, Nayu Liu, Zhicong Lu, Qing Liu, Linhao Zhang, Jiang Zhong, and Yanfeng Hu. 2025. U-mere: Unconstrained multimodal entity and relation extraction with collaborative modeling and order-sensitive optimization. In *Proceedings of the 33rd ACM International Conference on Multimedia*, pages 4349–4358.
- Zhehan Kan, Yanlin Liu, Kun Yin, Xinghua Jiang, Xin Li, Haoyu Cao, Yinsong Liu, Deqiang Jiang, Xing Sun, Qingmin Liao, and 1 others. 2025. Taco: Think-answer consistency for optimized long-chain reasoning and efficient data learning via reinforcement learning in lvlms. *arXiv preprint arXiv:2505.20777*.
- Ming Li, Jike Zhong, Shitian Zhao, Yuxiang Lai, Haoquan Zhang, Wang Bill Zhu, and Kaipeng Zhang. 2026. To think or not to think: A study of thinking in rule-based visual reinforcement fine-tuning. *Advances in Neural Information Processing Systems*, 38:167356–167407.
- Byeonggeuk Lim, Kyeonghyun Kim, JungMin Yun, and YoungBin Kim. 2026. Vg-cot: Towards trustworthy visual reasoning via grounded chain-of-thought. *arXiv preprint arXiv:2604.21396*.
- Ziyu Liu, Zeyi Sun, Yuhang Zang, Xiaoyi Dong, Yuhang Cao, Haodong Duan, Dahua Lin, and Jiaqi Wang. 2025. Visual-rft: Visual reinforcement fine-tuning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 2034–2044.

- Pan Lu, Hritik Bansal, Tony Xia, Jiacheng Liu, Chunyuan Li, Hannaneh Hajishirzi, Hao Cheng, Kai-Wei Chang, Michel Galley, and Jianfeng Gao. 2024a. Mathvista: Evaluating mathematical reasoning of foundation models in visual contexts. In *International Conference on Learning Representations*, volume 2024, pages 23439–23554.
- Zhicong Lu, Li Jin, Peiguang Li, Yu Tian, Linhao Zhang, Sirui Wang, Guangluan Xu, Changyuan Tian, and Xunliang Cai. 2024b. Rethinking the reversal curse of llms: a prescription from human knowledge reversal. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 7518–7530.
- Zhicong Lu, Zichuan Lin, Wei Jia, Changyuan Tian, Deheng Ye, Peiguang Li, Li Jin, Nayu Liu, Guangluan Xu, and Wei Feng. 2026. Hisr: Hindsight information modulated segmental process rewards for multi-turn agentic reinforcement learning. *arXiv preprint arXiv:2603.18683*.
- Zhicong Lu, Changyuan Tian, Peiguang Li, Peiguang Li, Li Jin, Sirui Wang, Wei Jia, Ying Shen, and Guangluan Xu. 2025. Piper: Benchmarking and prompting event reasoning boundary of llms via debiasing-distillation enhanced tuning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 28591–28613.
- Yunze Man, De-An Huang, Guilin Liu, Shiwei Sheng, Shilong Liu, Liang-Yan Gui, Jan Kautz, Yu-Xiong Wang, and Zhiding Yu. 2025. Argus: Vision-centric reasoning with grounded chain-of-thought. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 14268–14280.
- OpenAI. 2025. [Gpt-5 system card](#). Technical report, OpenAI.
- Arijit Ray, Jiafei Duan, Reuben Tan, Dina Bashkirova, Rose Hendrix, Kiana Ehsani, Aniruddha Kembhavi, Bryan A Plummer, Ranjay Krishna, Kuo-Hao Zeng, and 1 others. 2024. Sat: Spatial aptitude training for multimodal language models. *arXiv preprint arXiv:2412.07755*, 3.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, and 1 others. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.
- Wenhao Shi, Zhiqiang Hu, Yi Bin, Junhua Liu, Yang Yang, See Kiong Ng, Lidong Bing, and Roy Ka-Wei Lee. 2024. Math-llava: Bootstrapping mathematical reasoning for multimodal large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 4663–4680.
- Huajie Tan, Yuheng Ji, Xiaoshuai Hao, Xiansheng Chen, Pengwei Wang, Zhongyuan Wang, and Shanghang Zhang. 2026. Reason-rft: Reinforcement fine-tuning for visual reasoning of vision language models. *Advances in neural information processing systems*, 38:5772–5822.
- Changyuan Tian, Zhicong Lu, Huaxing Liu, Xiang Wang, Shuai Li, Yu Chen, Wenqian Lv, Zichuan Lin, Juncheng Diao, and Deheng Ye. 2026a. Faithfulmr1: Faithful multimodal reasoning via anchoring and reinforcing visual attention. *arXiv preprint arXiv:2605.22072*.
- Changyuan Tian, Zhicong Lu, Shuang Qian, Nayu Liu, Peiguang Li, Li Jin, Leiyi Hu, Zhizhao Zeng, Sirui Wang, Ke Zeng, and 1 others. 2026b. Rectify evaluation preference: Improving llms’ critique on math reasoning via perplexity-aware reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 33241–33249.
- Shengbang Tong, Ellis Brown, Penghao Wu, Sanghyun Woo, Manoj Middepogu, Sai C Akula, Jihan Yang, Shusheng Yang, Adithya Iyer, Xichen Pan, and 1 others. 2024. Cambrian-1: A fully open, vision-centric exploration of multimodal llms. *Advances in Neural Information Processing Systems*, 37:87310–87356.
- Ke Wang, Junting Pan, Weikang Shi, Zimu Lu, Houxing Ren, Aojun Zhou, Mingjie Zhan, and Hongsheng Li. 2024a. Measuring multimodal mathematical reasoning with math-vision dataset. *Advances in Neural Information Processing Systems*, 37:95095–95169.
- Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, and 1 others. 2024b. Qwen2-vl: Enhancing vision-language model’s perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*.
- Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, Alexis Gallagher, Raja Biswas, Faisal Ladhak, Tom Aarsen, and 1 others. 2025. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2526–2547.
- Zhuosheng Zhang, Aston Zhang, Mu Li, Hai Zhao, George Karypis, and Alex Smola. 2023. Multimodal chain-of-thought reasoning in language models. *arXiv preprint arXiv:2302.00923*.

A Implementation Details

We train the consistency reward model with ModernBERT-large and implement CORA, both using bf16 precision. The rollout temperature is set to 1.0 for RL training. The standard GRPO baselines are reproduced on the same training data with answer-correctness and format rewards. All

training runs are conducted on 16 NVIDIA A100 GPUs.

For spatial perception, we fine-tune on SAT for 2 epochs and evaluate on CVBench; the maximum prompt and response lengths are both set to 1024. For multimodal mathematical reasoning, we fine-tune on Math-40K for 1 epoch and evaluate on MathVista and MathVision; the maximum prompt and response lengths are set to 4096 and 512, respectively. For visual puzzle reasoning, we fine-tune for 2 epochs on the generated PuzzleVQA training set following the generation pipeline used by Li et al. (2026), and evaluate on PuzzleVQA and AlgoPuzzleVQA; the maximum prompt and response lengths are both set to 1024.

For the consistency reward, we adopt two types of scheduling strategies: warmup and decay. Warmup means that the consistency reward is temporarily disabled at the beginning of training, allowing the model to first learn basic task-solving behavior from the task reward. Decay means that, after the consistency reward is activated, its weight is gradually reduced as training proceeds, thereby preventing this signal from dominating the later training stage. The three datasets use different scheduling schemes according to their task characteristics. During optimization, the advantages of the task-reward branch and the consistency-reward branch are computed separately, where α and β denote the weights of the two branches, respectively.

Regarding Qwen2.5-VL-7B, we adopt both warmup and decay on the SAT dataset. The consistency reward is disabled for the first 200 steps. After that, its initial weight is set to 0.3 and decays exponentially with a half-life of 40 steps. The minimum weight threshold is set to 0.05. Once the weight decays below this threshold, the consistency reward is no longer applied. For advantage computation, the task and consistency branches are weighted by α and β with values of 1.0 and 0.05, respectively. On the Math-40K dataset, we only apply decay. The initial weight of the consistency reward is set to 1.0, the half-life is set to 183 steps, and the minimum weight threshold is set to 0.05. For advantage computation, the task and consistency branch weights are 1.0 and 0.5, respectively. On the PuzzleVQA dataset, we only apply warmup. Specifically, the consistency reward weight is set to 0 for the first 400 steps. After the warmup stage, it is fixed at 1.0 and kept unchanged throughout the remaining training process. For advantage computation, the task and consistency branch weights are

Item	Setting
Base model	ModernBERT-large
Number of labels	2
Maximum sequence length	2048
Batch size	16
Epochs	3
Learning rate	2×10^{-5}

Table 4: Training configuration for the binary consistency reward model.

Metric	Score
Overall accuracy	0.8734
Macro F1	0.8733
Weighted F1	0.8733

Table 5: Overall the test set of the consistency reward model.

1.0 and 0.5, respectively.

B Consistency Reward Model Details

We train a binary consistency reward model to judge whether a response’s reasoning trace supports its final answer. The classifier uses answerdotai/ModernBERT-large as the backbone and predicts two labels: *consistent* and *inconsistent*. Table 4 summarizes the training configuration, and Table 6 reports the held-out evaluation results used by our consistency reward.

C Comparison with GPT-4o-mini Reward Model

To verify whether CORA’s effectiveness depends on using a large language model as the online consistency reward, we conduct an additional comparison by replacing our lightweight CRM with GPT-4o-mini as the consistency reward model. We keep the RLVR training setting the same as CORA and run the experiment on Qwen2.5-VL-7B, using SAT for RLVR training and CVBench for evaluation. Due to the high computational and monetary cost of invoking the closed-source LLM as the reward model during RL rollouts, we conduct this comparison only on this representative setting.

As shown in Table 7, GPT-RM obtains a lower inconsistency rate, but its answer accuracy is lower than CORA by 2.92 points. To further analyze the reason behind this gap, we use GPT-5 to perform the judgment of the consistency between each generated thinking trace and its final answer after inference, and divide the responses according to both consistency and answer correctness. The

Source	n	Acc.	Macro F1	R(con)	R(inc)
Math	10014	0.8097	0.8090	0.8613	0.7571
Puzzle	6037	0.9730	0.9730	0.9707	0.9753
Sat	2036	0.8919	0.8919	0.9108	0.8742

Table 6: Consistency reward model performance by source dataset. R(con) and R(inc) denote recall for the consistent and inconsistent classes, respectively.

Type	Metric	CORA	GPT-RM	Δ
Overall	Accuracy $\uparrow$	78.24%	75.32%	+2.92
	Inconsistency rate $\downarrow$	7.36%	5.95%	+1.41
Detailed analysis	CC: consistent and correct $\uparrow$	72.25%	71.38%	+0.87
	CW: consistent but wrong $\downarrow$	18.95%	22.14%	-3.18
	IC: inconsistent but correct	5.99%	3.94%	+2.05
	IW: inconsistent and wrong $\downarrow$	2.81%	2.54%	+0.27

Table 7: Comparison between CORA and the GPT-4o-mini reward model on CVBench after SAT training with Qwen2.5-VL-7B. GPT-RM denotes replacing our lightweight CRM with GPT-4o-mini as the consistency reward model. CC denotes consistent and correct, and CW denotes consistent but wrong. Δ is CORA minus GPT-RM.

results show that GPT-RM has a larger fraction of consistent-but-wrong responses: GPT-RM produces 22.14% CW samples, while CORA reduces this group to 18.95%. Meanwhile, CORA produces more consistent-and-correct responses than GPT-RM. This indicates that the lower inconsistency rate of GPT-RM is partly caused by generating responses whose thinking and answer are consistently wrong, rather than by improving task correctness. The possible reason is that a closed-source LLM judge may rely on its own prior knowledge when scoring the rollout, which does not necessarily provide a stable reward signal for aligning the model’s internal thinking with its answer. In contrast, our lightweight CRM is explicitly trained to estimate thinking-answer consistency and can be plugged into RLVR training after a single efficient training stage, providing a lower-cost and faster reward model while preserving stronger answer accuracy.

D Prompt Templates for Consistency Annotation

Tables 8–11 report the API prompts used for thinking-answer consistency annotation on the training-stage corpus and the evaluation-stage corpus.

E Accuracy and Consistency Reward Curves during RLVR

Figure 5 compares the accuracy reward trajectories of the GRPO baseline and CORA during RLVR on different training datasets. The results show that introducing the consistency reward generally improves the answer-level reward obtained during training. We also observe that the effect of CORA is task-dependent. In particular, the improvement is more pronounced on PuzzleVQA than on SAT. We attribute this to the fact that PuzzleVQA relies more heavily on intermediate reasoning, where process-level supervision from CORA can provide a stronger and more useful training signal.

Figure 6 presents the consistency reward trajectory during training. The curve shows that the introduced consistency reward provides an effective supervisory signal for aligning the reasoning process with the final answer.

F Additional Case Studies

We provide supplementary case studies from CVBench, MathVision, MathVista, and PuzzleVQA. Each case compares the same Qwen2.5-VL-7B-Instruct backbone before and after our CORA. Red text marks the baseline reasoning step that contradicts its final answer, while green text marks the corrected thinking step in our model.

Across the cases, the baseline often reaches a final `<answer>` that is not supported by its own `<think>` trace: it misreads set containment, reverses a depth relation, rejects all cube configurations before selecting one, confuses equal and supplementary angles, or concludes the wrong color pattern. Our model does not merely change the final answer tag; it revises the decisive reasoning step so that the visual observation, intermediate conclusion, and final answer become aligned. These examples illustrate that the consistency reward improves the semantic coupling between reasoning and answering across spatial perception, geometric reasoning, and visual pattern completion.

Usage	SAT training-rollout consistency annotation
Prompt	I will give you a SAT (spatial reasoning) question, its options, and a model’s response. The model response contains a thinking process in <think>...</think> tags and a final answer in <answer>...</answer> tags. SAT questions usually involve counting objects, judging spatial relations (left/right, above/below, in-front/behind), estimating depth/distance from camera, or identifying which marked object is closer/farther. Your task: (1) read the thinking process and determine the final conclusion the reasoning arrived at, focusing on the conclusion sentence(s); (2) read the <answer> tag and copy the value the model committed to; (3) judge whether the thinking conclusion and the answer tag are consistent. Important rules: think_conclusion is the final value the thinking process actually concludes with. Copy explicit final values even if they are outside the option set. For yes/no questions, infer yes/no if the reasoning implies an affirmative or negative conclusion. Pay attention to negation and perspective equivalence. Output None if the thinking is generic or lists candidates without choosing one. think_in_options is Yes, No, or None; answer_tag copies the value inside <answer> even if outside the options; answer_in_options is Yes, No, or None. consistency is Yes if the two values refer to the same value after normalization, No if they differ, and None only when either side is missing. Output exactly five lines: think_conclusion: <value, option, or None>; think_in_options: <Yes/No/None>; answer_tag: <value, option, or None>; answer_in_options: <Yes/No/None>; consistency: <Yes/No/None>. <i>Few-shot SAT examples are inserted here.</i> Now analyze this sample: Question: {question}; Options: {options}; Model response: {response}.

Table 8: Prompt template for annotating think-answer consistency in SAT training rollouts. The full script inserts SAT-specific few-shot examples covering counting, spatial relations, yes/no inference, perspective equivalence, negation, and answers outside the option set.

Usage	Math-40K training-rollout consistency annotation
Prompt	I will give you a math/VQA question, its question type, its options (if any), and a model’s response. The model response contains a thinking process in <think>...</think> tags and a final answer in <answer>...</answer> tags. Question types in this dataset: letter_choice, where the question has explicit choices such as (A)–(D), and open_ended, where there is no fixed option set. For letter_choice, the reasoning may state the option text instead of the letter; treat these as the same option. For open_ended, both thinking and answer can be arbitrary text or numbers. Your task: (1) read the thinking process and determine the final conclusion; (2) read the <answer> tag and copy the committed value; (3) judge whether the thinking conclusion and answer tag are consistent. Important rules: for letter_choice, think_conclusion can be a letter, option text, or a raw value outside the options; for open_ended, copy the final number, color, or identification phrase. Infer implicit yes/no conclusions, handle negation, and output None when the reasoning has no concrete conclusion. For letter_choice, think_in_options and answer_in_options are Yes, No, or None; for open_ended, they are always N/A. consistency is Yes when both sides refer to the same value after normalization or semantic equivalence, No when they differ, and None only when either side is missing. Output exactly five lines with no extra text: think_conclusion: <value or None>; think_in_options: <Yes/No/None/N/A>; answer_tag: <value or None>; answer_in_options: <Yes/No/None/N/A>; consistency: <Yes/No/None>. <i>Few-shot Math-40K examples are inserted here.</i> Now analyze this sample: Question type: {question_type}; Options: {options}; Question: {question}; Model response: {response}.

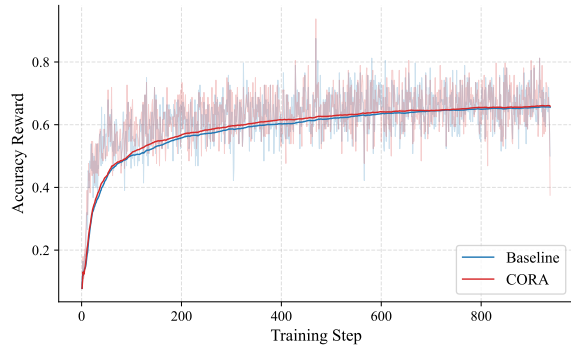
Table 9: Prompt template for annotating think-answer consistency in Math-40K training rollouts. The prompt supports both multiple-choice and open-ended questions and keeps out-of-option conclusions as raw values.

Usage	PuzzleVQA training-rollout consistency annotation
Prompt	I will give you a puzzle question, its options, and a model’s response. The model response contains a thinking process in <think>...</think> tags and a final answer in <answer>...</answer> tags. The puzzles ask the model to find a missing element (number, color, shape, or size) in a visual pattern. The thinking process is often short and may not actually reach a conclusion that names one of the options. Your task: (1) read the thinking process and determine the final conclusion; (2) read the <answer> tag and copy the committed value; (3) judge whether the thinking conclusion and answer tag are consistent. Important rules: think_conclusion is the final value the thinking actually concludes with. Copy explicit values even if they are outside the option set; output None if the thinking is generic or only lists candidates. think_in_options is Yes, No, or None. answer_tag copies the value inside <answer> as-is, even if it is outside the options; answer_in_options is Yes, No, or None. consistency is Yes if both sides refer to the same normalized value, including when both are outside the options; No if they refer to different values; and None only when either side is missing. Compound colors such as “light orange” and “orange” must be treated as different values. Output exactly five lines with no extra text: think_conclusion: <value or None>; think_in_options: <Yes/No/None>; answer_tag: <value or None>; answer_in_options: <Yes/No/None>; consistency: <Yes/No/None>. <i>Few-shot PuzzleVQA examples are inserted here.</i> Now analyze this sample: Question: {question}; Options: {options}; Model response: {response}.

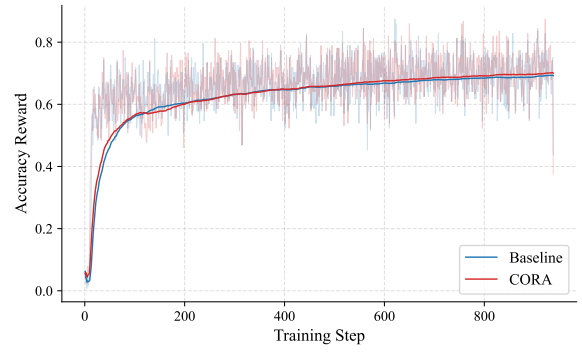
Table 10: Prompt template for annotating think-answer consistency in PuzzleVQA training rollouts. The prompt explicitly preserves answers outside the option set and distinguishes compound colors.

Usage	Evaluation-set consistency extraction
Prompt	I will give you a question and a model response. The model response is in the form <think>...</think><answer>...</answer>. Your job is to extract two things and judge their consistency, looking only at the literal content of <think>...</think> and <answer>...</answer>. Do not use outside knowledge to guess what the correct answer should be. Do not bias your extraction toward any expected answer; only report what the model literally wrote. Specifically: (1) answer in thinking tag: extract what the thinking process commits to as a final choice. If it does not explicitly commit to one of the listed options, output None. (2) answer in answer tag: extract whatever letter/value is literally inside <answer>...</answer>. If it is a single letter, expand it to its full option text using the option list; if it is already in “(X) value” form, keep it. (3) consistency with answer tag: output Yes if the two extracted values refer to the same option/value, No if they differ, and None if the thinking answer is None. Output exactly these three lines: answer in thinking tag;; answer in answer tag;; consistency with answer tag: <Yes/No/None>. <i>Dataset-specific few-shot examples for CVBench, MathVision, MathVista, PuzzleVQA, or AlgoPuzzleVQA are inserted here.</i> Question: {question}; Please choose one from list: {option_text}; model response: {response}.

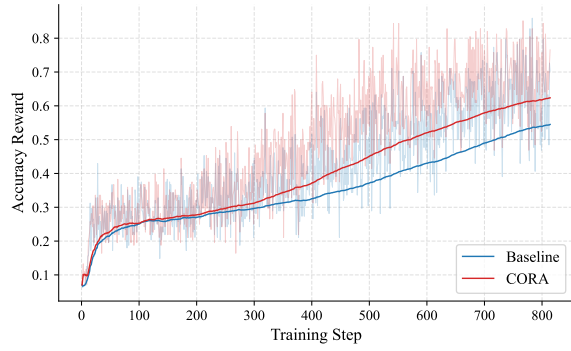
Table 11: Prompt template for extracting think-answer consistency on held-out evaluation benchmarks. Unlike the earlier prototype, this fixed prompt intentionally does not include the ground-truth answer, preventing leakage when extracting the answer committed to by <answer>.



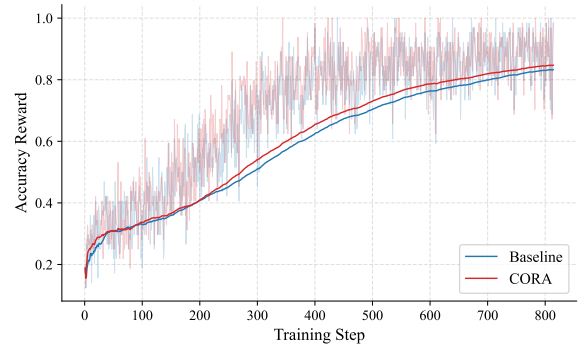
(a) SAT, Qwen2-VL-2B



(b) SAT, Qwen2.5-VL-3B



(c) PuzzleVQA, Qwen2-VL-2B



(d) PuzzleVQA, Qwen2-VL-7B

Figure 5: Accuracy reward trajectories during RLVR. We compare the GRPO baseline and CORA on SAT and PuzzleVQA. Light curves show raw logged rewards, and dark curves show smoothed trends.

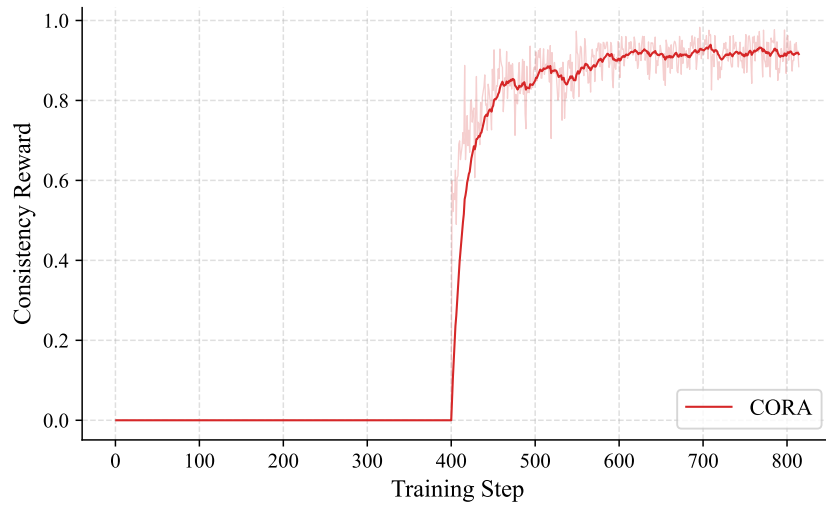
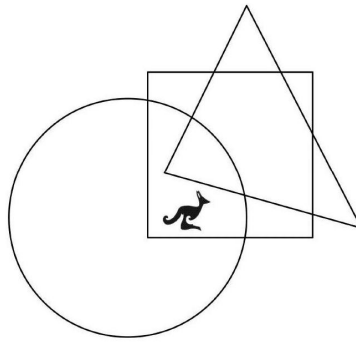


Figure 6: Consistency reward trajectory during RLVR on PuzzleVQA with Qwen2-VL-7B. The curve shows that CORA provides a non-trivial process-level supervision signal for aligning the reasoning process with the final answer.

F.1 Case Study on MathVision



Question: Where is the Kangaroo? **Options:** (A) In the circle and in the triangle but not in the square., (B) In the circle and in the square but not in the triangle., (C) In the triangle and in the square but not in the circle., (D) In the circle but in neither the square or the triangle., (E) In the square but in neither the circle or the triangle. **Ground truth:** (B) In the circle and in the square but not in the triangle.

Baseline output.

<think>

To determine where the kangaroo is located, we need to analyze the image step by step:

1. The kangaroo is inside a square. 2. The square is partially inside a triangle. 3. The triangle is partially inside a circle.

From this analysis: - The kangaroo is definitely in the square.

- The kangaroo is also in the triangle because the square is inside the triangle.

- The kangaroo is not in the circle because the square is outside the circle.

Therefore, the kangaroo is in the square and the triangle but not in the circle.

</think>

<answer> B </answer>

Our method output.

<think>

The image shows a kangaroo inside a square that is also inside a circle. The kangaroo is not within the triangle.

- The kangaroo is clearly inside the square.

- The square is inside the circle.

- The kangaroo is not inside the triangle.

Therefore, the kangaroo is in the circle and in the square but not in the triangle.

</think>

<answer>B</answer>

F.2 Case Study on CVBench



Question: Which object is closer to the camera taking this photo, the traffic cone (highlighted by a red box) or the trailer (highlighted by a blue box)? **Options:** (A) traffic cone, (B) trailer **Ground truth:** (A) traffic cone

Baseline output.

<think>

To determine which object is closer to the camera, we need to consider the perspective and the size of the objects in the image.

1. The traffic cone is located on the left side of the image, near the bottom edge. It appears smaller compared to the trailer, indicating it is farther away from the camera.

2. The trailer is highlighted by a blue box and is positioned further back in the scene, closer to the center of the image. It appears larger than the traffic cone, suggesting it is closer to the camera.

Based on these observations, the trailer is closer to the camera.

</think>

<answer>(A) traffic cone</answer>

Our method output.

<think>

To determine which object is closer to the camera, we need to consider the perspective and relative size of the objects in the image.

1. The traffic cone is located on the left side of the image, near the bottom edge, and appears smaller compared to the trailer.

2. The trailer is highlighted by a blue box and is positioned further back in the scene, closer to the center of the image.

3. The traffic cone's size and position suggest it is closer to the camera than the trailer.

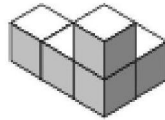
Based on these observations, the traffic cone is closer to the camera.

</think>

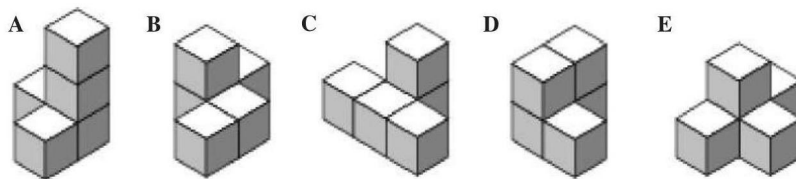
<answer>(A) traffic cone</answer>

F.3 Case Study on MathVision

<image1>:



<image2>:



Question: Which of the “buildings” A-E, each consisting of 5 cubes, cannot be obtained from the building on the right, if you are allowed to move only one cube? **Options:** (A) A, (B) B, (C) C, (D) D, (E) E

Ground truth: (C) C

Baseline output.

<think>

To determine which of the buildings A-E cannot be obtained by moving only one cube from the building on the right, we need to analyze the structure of the building on the right and compare it with the structures of the buildings A-E.

The building on the right consists of 5 cubes arranged in a specific pattern. We need to check if any of the buildings A-E can be formed by moving exactly one cube from this building.

Let's examine each option:

- A: This building has a different arrangement of cubes compared to the original building. It appears that the top-left cube has been moved to the bottom-right position.

- B: This building also has a different arrangement. The top-left cube seems to have been moved to the bottom-left position.

- C: This building has a different arrangement as well. The top-left cube appears to have been moved to the bottom-center position.

- D: This building has a different arrangement. The top-left cube seems to have been moved to the bottom-right position.

- E: This building has a different arrangement. The top-left cube appears to have been moved to the bottom-center position.

Since all the buildings A-E have a different arrangement of cubes compared to the original building, none of them can be obtained by moving only one cube from the building on the right.

</think>

<answer>A</answer>

Our method output.

<think>

To determine which of the buildings A-E cannot be obtained by moving only one cube from the given building, we need to analyze the structure of the original building and compare it with the structures of the options A-E.

The original building has a specific arrangement of 5 cubes. We need to check if any of the buildings A-E can be formed by moving just one cube from this original building.

Let's examine each option:

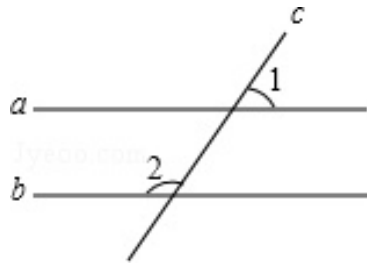
- A: This building looks like it could be formed by moving one cube from the original building.
- B: This building also seems possible by moving one cube.
- C: This building appears to be a different configuration that cannot be formed by moving just one cube from the original building.
- D: This building looks like it could be formed by moving one cube.
- E: This building also seems possible by moving one cube.

From the analysis, the only building that cannot be formed by moving just one cube from the original building is C. Therefore, the answer is C.

</think>

<answer>C</answer>

F.4 Case Study on MathVista



Question: As shown in the figure, the parallel lines a and b are intercepted by the straight line c . If $\angle 1 = 50^\circ$, then the degree of $\angle 2$ is () **Options:** (A) 150° , (B) 130° , (C) 110° , (D) 100° **Ground truth:** (B) 130°

Baseline output.

<think> Since lines a and b are parallel, and line c intersects them, the angles formed are corresponding angles. Therefore, $\angle 1$ and $\angle 2$ are corresponding angles. The property of corresponding angles states that if two parallel lines are cut by a transversal, the corresponding angles are equal. Hence, $\angle 2 = \angle 1$.</think>

<answer>B</answer>

Our method output.

<think> Since lines a and b are parallel, and line c intersects them, the angles formed are corresponding angles. Therefore, $\angle 1$ and $\angle 2$ are supplementary angles (they add up to 180°). Given that $\angle 1 = 50^\circ$, we can calculate $\angle 2$ as follows:

$$\angle 2 = 180^\circ - \angle 1$$

$$\angle 2 = 180^\circ - 50^\circ$$

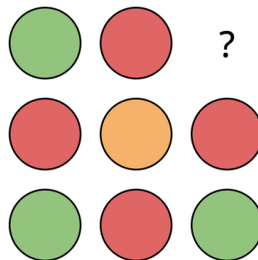
$$\angle 2 = 130^\circ$$

So, the degree of $\angle 2$ is 130° .

Therefore, the correct answer is (B) 130° .</think>

<answer>(B)</answer>

F.5 Case Study on PuzzleVQA



Question: What is the color of the missing part denoted with a question mark? **Options:** (A) purple, (B) orange, (C) red, (D) green **Ground truth:** (D) green

Baseline output.

<think>

To determine the color of the missing part, let's analyze the pattern in the image:

1. The top row has two circles: one green and one red.
2. The middle row has three circles: two red and one orange.
3. The bottom row has three circles: two green and one red.

The pattern seems to be alternating colors in each row. In the first row, we have green and red. In the second row, we have two reds and one orange, which doesn't follow the same alternating pattern as the first row. However, if we look at the third row, it follows the same alternating pattern as the first row: two green and one red.

Given this pattern, the missing circle should be red to maintain the alternating pattern of green and red in the rows.

Therefore, the color of the missing part denoted with a question mark is red.

</think>

<answer>D</answer>

Our method output.

<think> To determine the color of the missing part, let's analyze the pattern in the image:

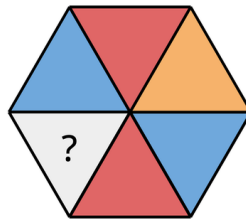
1. The top row has two circles: one green and one red.
2. The middle row has three circles: two red and one orange.
3. The bottom row has three circles: two green and one red.

The pattern seems to be that each row alternates between two colors. In the first row, it starts with green and ends with red. In the second row, it starts with red and ends with orange. Following this pattern, the third row should start with green and end with red.

Since the question mark is next to a red circle and the pattern suggests it should be green to continue the alternating sequence, the missing part should be green. Therefore, the color of the missing part is (D) green.

</think>

<answer>(D) green</answer>

F.6 Case Study on PuzzleVQA

Question: What is the missing color of the part denoted with a question mark? **Options:** (A) green, (B) purple, (C) red, (D) orange **Ground truth:** (D) orange (letter: D)

Baseline output.

<think>

To determine the missing color, let's analyze the pattern in the hexagon:

1. The hexagon is divided into six triangular sections.
2. Each section appears to have a unique color.
3. The colors in the adjacent sections seem to alternate in a specific way.

Looking at the sections:

- Top left: Blue
- Top middle: Red
- Top right: Orange
- Bottom right: Blue
- Bottom middle: Red
- Bottom left: White

The pattern seems to be that each color alternates with the next color in a clockwise direction.

If we follow this pattern: - After Orange (top right), the next color should be Red (bottom middle).

Therefore, the missing color for the section denoted with a question mark should be Red.

</think>

<answer>D</answer>

Our method output.

<think>

To determine the missing color, let's analyze the pattern in the hexagon:

1. The hexagon is divided into six triangular sections.

2. Each section appears to have a different color.
3. The colors seem to alternate in a specific sequence.

Looking at the sections:

- Top left: Blue
- Top middle: Red
- Top right: Orange
- Bottom right: Blue
- Bottom middle: Red

The bottom left section is the one with the question mark. Observing the sequence, it seems that the colors are repeating in a pattern: Blue, Red, Orange, Blue, Red.

Following this pattern, the next color after Red should be Orange. Therefore, the missing color is (D) orange.

</think>

<answer>(D) orange</answer>